PLAN DE MANTENIMIENTO DE SOFTWARE

CanchaFácil

Código del Documento: PMS-[Año]-[Versión]
Versión: 1.0
Fecha de Creación: [Fecha]
Responsable del Documento: [Nombre/Cargo]
Estado: [Borrador]

1. INTRODUCCIÓN Y OBJETIVOS

1.1. Propósito

Este documento establece las estrategias, procedimientos y responsabilidades para realizar el mantenimiento del sistema CanchaFácil después de su implementación. Su propósito es garantizar el correcto funcionamiento de la plataforma, corregir errores, proteger la información y mejorar continuamente sus funcionalidades.

1.2. Alcance

El plan de mantenimiento comprende los siguientes componentes:

- Aplicación web desarrollada con Django.
- Módulo de administración de establecimientos y canchas.
- Módulo de usuarios y autenticación.
- Módulo de disponibilidad y reservas.
- Módulo de pagos y registro de abonos.
- Interfaz gráfica desarrollada con HTML, CSS, Bootstrap y JavaScript.
- Base de datos SQLite.

Excluye:

- Mantenimiento físico de computadores y servidores.
- Administración de servicios de alojamiento externos.
- Soporte de infraestructura de terceros.
- Problemas relacionados con la conexión a internet del usuario.

2. INVENTARIO DE ACTIVOS DE SOFTWARE

Componente / Módulo	Tecnología / Framework	Versión Actual	Servidor / Entorno
Backend	Python / Django	Python 3.x / Django 4.x o 5.x	Desarrollo local
Frontend	HTML, CSS y JavaScript	HTML5 / CSS3	Navegador web
Framework visual	Bootstrap	5.3.3	Desarrollo local
Base de datos	SQLite	Versión incluida en el entorno	Local
Gestión de imágenes	Pillow	Según instalación	Desarrollo local
Editor de código	Visual Studio Code	Según instalación	Windows

3. TIPOS DE MANTENIMIENTO Y PROCEDIMIENTOS

3.1. Mantenimiento Correctivo (Resolución de fallas)

- **Activación:** Se inicia cuando un usuario o administrador reporta un error en el sistema, como problemas al registrar canchas, consultar disponibilidad o realizar reservas.
- **Procedimiento:**

1. Identificar y registrar el error.
2. Reproducir el problema en el entorno de desarrollo.
3. Identificar la causa del defecto.
4. Corregir el código o la configuración.
5. Realizar pruebas para comprobar la solución.
6. Verificar que las demás funcionalidades continúen funcionando correctamente.

3.2. Mantenimiento Preventivo (Acciones proactivas)

- **Actualizaciones de seguridad:** Revisar periódicamente las versiones de Django, Python y las librerías utilizadas.
- **Respaldos:** Realizar copias de seguridad de la base de datos y de los archivos importantes del proyecto.
- **Optimización:** Revisar consultas a la base de datos, eliminar código innecesario y mejorar los tiempos de respuesta.

- **Revisión de errores:** Consultar los registros de ejecución para identificar posibles fallos antes de que afecten a los usuarios.

3.3. Mantenimiento Adaptativo y Perfectivo (Evolución)

- **Adaptativo:** Modificar el sistema cuando existan cambios en las versiones de Python, Django, navegadores o herramientas utilizadas.
- **Perfectivo:** Agregar mejoras a la interfaz, optimizar el proceso de reservas, mejorar las validaciones y facilitar la administración de las canchas.

4. NIVELES DE PRIORIDAD Y SLA (ACUERDOS DE NIVEL DE SERVICIO)

Severidad	Descripción	Tiempo de Respuesta	Tiempo de Resolución
Crítica (P1)	El sistema no permite ingresar o realizar las operaciones principales.	Menos de 1 hora	Menos de 8 horas
Alta (P2)	Una función importante, como las reservas, presenta fallos.	Menos de 4 horas	Menos de 24 horas
Media (P3)	Una funcionalidad secundaria presenta errores.	Menos de 24 horas	Menos de 48 horas
Baja (P4)	Errores visuales o sugerencias de mejora.	Menos de 48 horas	Próxima actualización

5. ROLES Y RESPONSABILIDADES

- **Líder del proyecto:** Organiza las actividades de mantenimiento, establece prioridades y revisa las mejoras propuestas.
- **Administrador del sistema:** Gestiona los usuarios, establecimientos, canchas y la información registrada en la plataforma.
- **Equipo de desarrollo:** Identifica y corrige errores, realiza pruebas y desarrolla nuevas funcionalidades.

- **Usuario final:** Reporta errores, informa problemas encontrados y proporciona sugerencias para mejorar el sistema.

6. CRONOGRAMA DE ACTIVIDADES PREVENTIVAS

Actividad	Frecuencia	Responsable	Ventana de Mantenimiento
Revisión de errores del sistema	Semanal	Desarrollador	Durante el mantenimiento
Copia de seguridad de la base de datos	Diaria o antes de cambios importantes	Desarrollador	Antes de realizar modificaciones
Actualización de librerías	Mensual	Desarrollador	Cuando sea necesario
Revisión de usuarios y permisos	Mensual	Administrador	Durante la revisión del sistema

7. CRITERIOS DE CONTROL DE CAMBIOS Y DESPLIEGUE

1. **Aislamiento:** Los cambios deben realizarse inicialmente en el entorno de desarrollo para evitar afectar el funcionamiento de la versión estable.
2. **Pruebas:** Antes de integrar una modificación, se deben realizar pruebas de las funcionalidades afectadas, especialmente usuarios, canchas y reservas.
3. **Respaldo:** Antes de realizar cambios importantes, se debe guardar una copia de seguridad de la base de datos y de los archivos del proyecto.
4. **Control de versiones:** Se recomienda utilizar Git y GitHub para registrar las modificaciones y facilitar la recuperación de versiones anteriores.
5. **Plan de retorno:** Si una actualización genera errores, se debe restaurar la versión anterior del código y recuperar el respaldo correspondiente cuando sea necesario.

8. MÉTRICAS E INDICADORES (KPIs)

- **Disponibilidad del sistema:** Porcentaje de tiempo durante el cual la plataforma se encuentra disponible para los usuarios, una vez esté desplegada en un servidor.
- **MTTR (Tiempo Medio de Reparación):** Tiempo promedio empleado para solucionar los errores reportados.
- **Cantidad de defectos encontrados:** Número de errores identificados durante las pruebas y el uso del sistema.
- **Tasa de errores corregidos:** Porcentaje de defectos solucionados frente al total de defectos registrados.
- **Pruebas exitosas:** Porcentaje de pruebas que finalizan correctamente después de realizar modificaciones.

9. SUGERENCIAS DE MEJORA Y CASOS FUERA DE ALCANCE

9.1. Sugerencias y Mejora Continua

Se propone recopilar las opiniones de los usuarios y administradores para identificar oportunidades de mejora en CanchaFácil. Algunas mejoras futuras pueden ser:

Implementar notificaciones automáticas sobre las reservas.

Mejorar la visualización de la disponibilidad de las canchas.

Ampliar las opciones de pago electrónico.

Incorporar reportes de ingresos y reservas.

Optimizar la experiencia de uso desde dispositivos móviles.

Mejorar los mecanismos de seguridad y recuperación de información.

9.2. Gestión de Excepciones (Casos No Atendibles)

Cuando se presente un problema que no pueda solucionarse directamente por el equipo de desarrollo, se deberá identificar su causa y determinar las acciones necesarias.

Algunos casos pueden ser:

Fallos en los servicios externos de pago.

Problemas de conectividad a internet.

Daños físicos en los equipos utilizados.

Incompatibilidad con versiones antiguas de navegadores.

Falta de recursos económicos o técnicos para implementar una mejora.

En estos casos, el incidente deberá documentarse y comunicarse al responsable del proyecto para evaluar alternativas, establecer prioridades y determinar si requiere apoyo externo o una modificación del alcance del sistema.